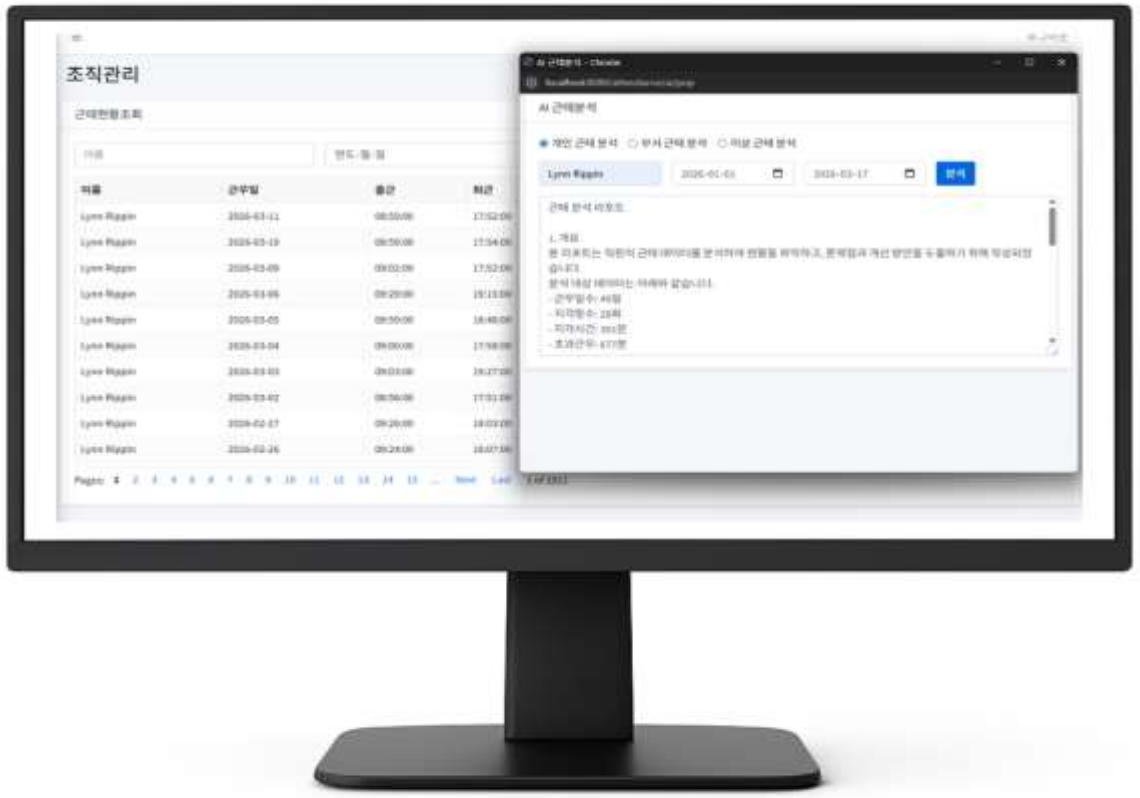
■ HRMS - AI 기반 근태 분석 기능을 제공하는 HRMS 인사관리 시스템

개발 기간
- 2026-02 ~ 2026-03

개발 인원
- 1명

기술 스택
- Java 17, Spring Boot 4.0.3
- Thymeleaf, JavaScript
- MySQL, Redis
- Docker, Nginx, Ubuntu, Jenkins
- Prometheus, Grafana

기타 도구
- Notion
- IntelliJ



결과물

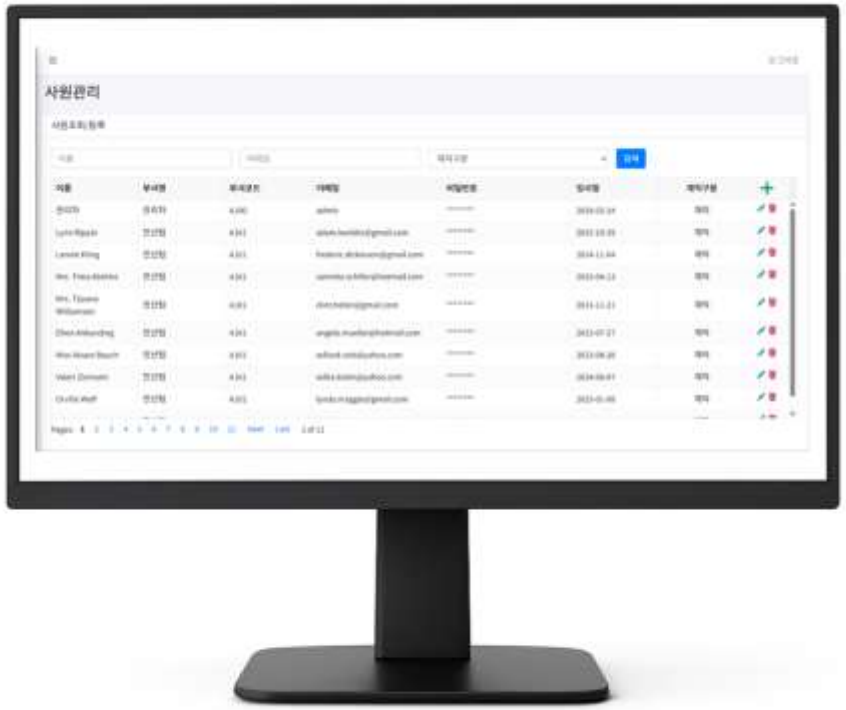
Github	https://github.com/wonjoong2/HRMS
서비스	https://myhrms.store
개발문서	https://productive-season-630.notion.site/HRMS-320d34243c84808ab567f69ceb8757be
ERD	https://github.com/wonjoong2/HRMS/blob/master/docs/HRMS_ERD.png

목차

1. **주요 기능 요약**
2. 배포
3. 어플리케이션
4. 주요 작업

■ HRMS – 주요 기능 요약

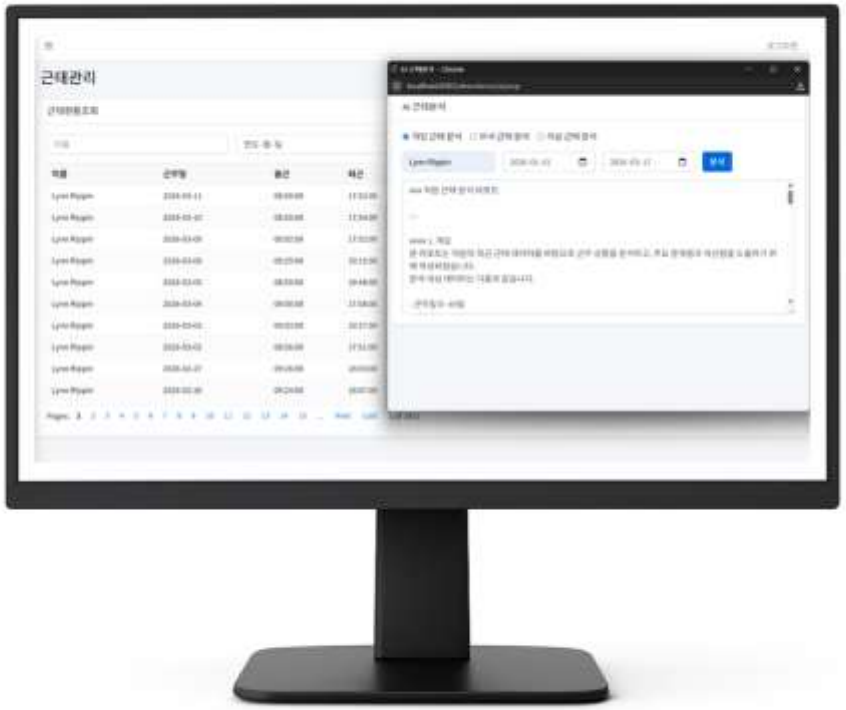
근태 대시보드



인사정보

- 사원 조회/등록

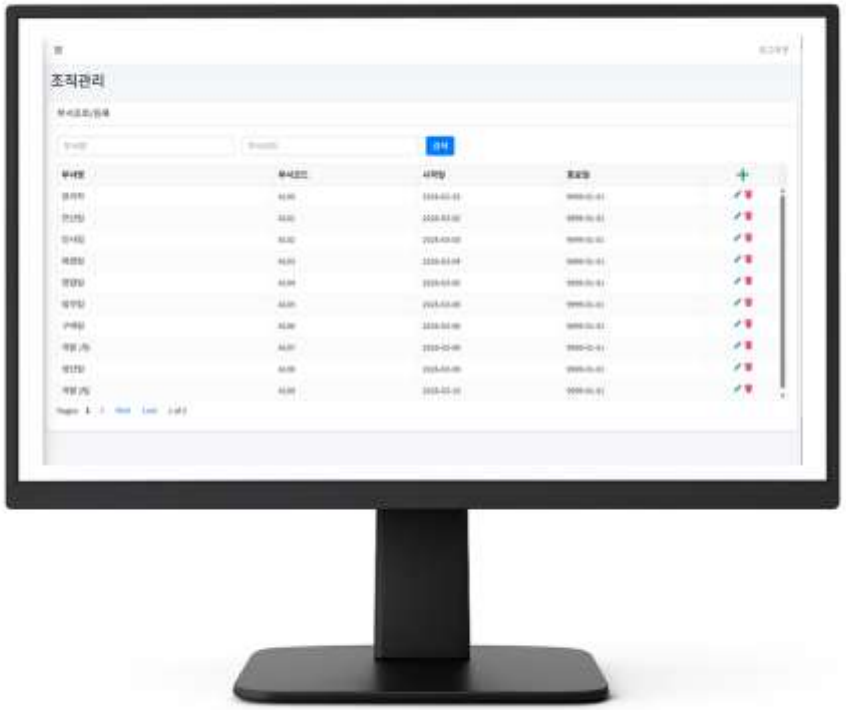
■ HRMS – 주요 기능 요약



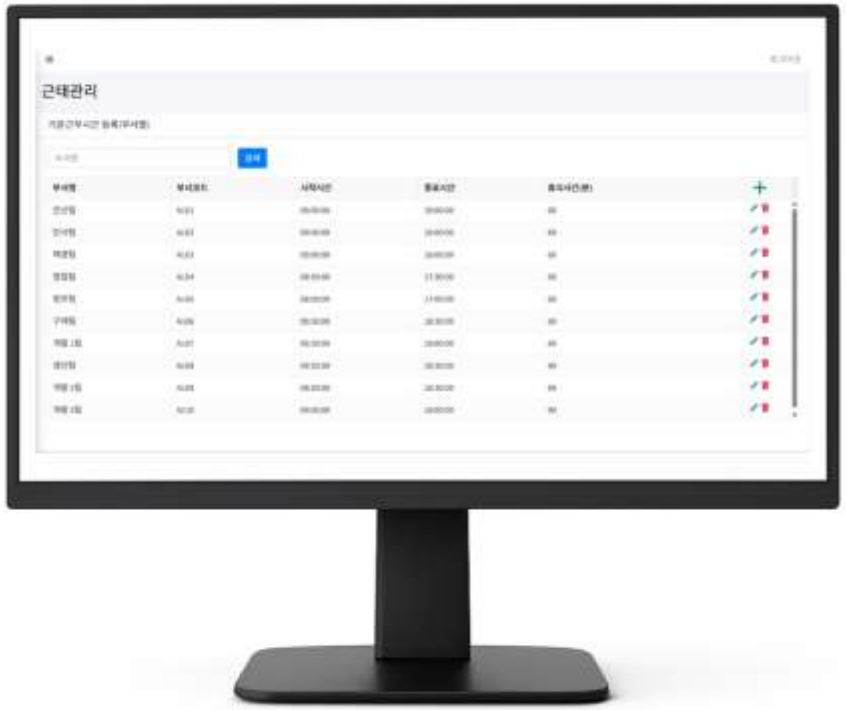
AI 근태분석

부서별 채팅

■ HRMS – 주요 기능 요약



조직관리
- 부서 조회/등록
- 부서이력조회



근태관리
- 근태현황조회
- 공휴일 등록
- 휴가신청현황
- 기준근무시간 설정

목차

1. 주요 기능 요약

2. 배포

- 프로젝트 아키텍처
- 백엔드 CI/CD

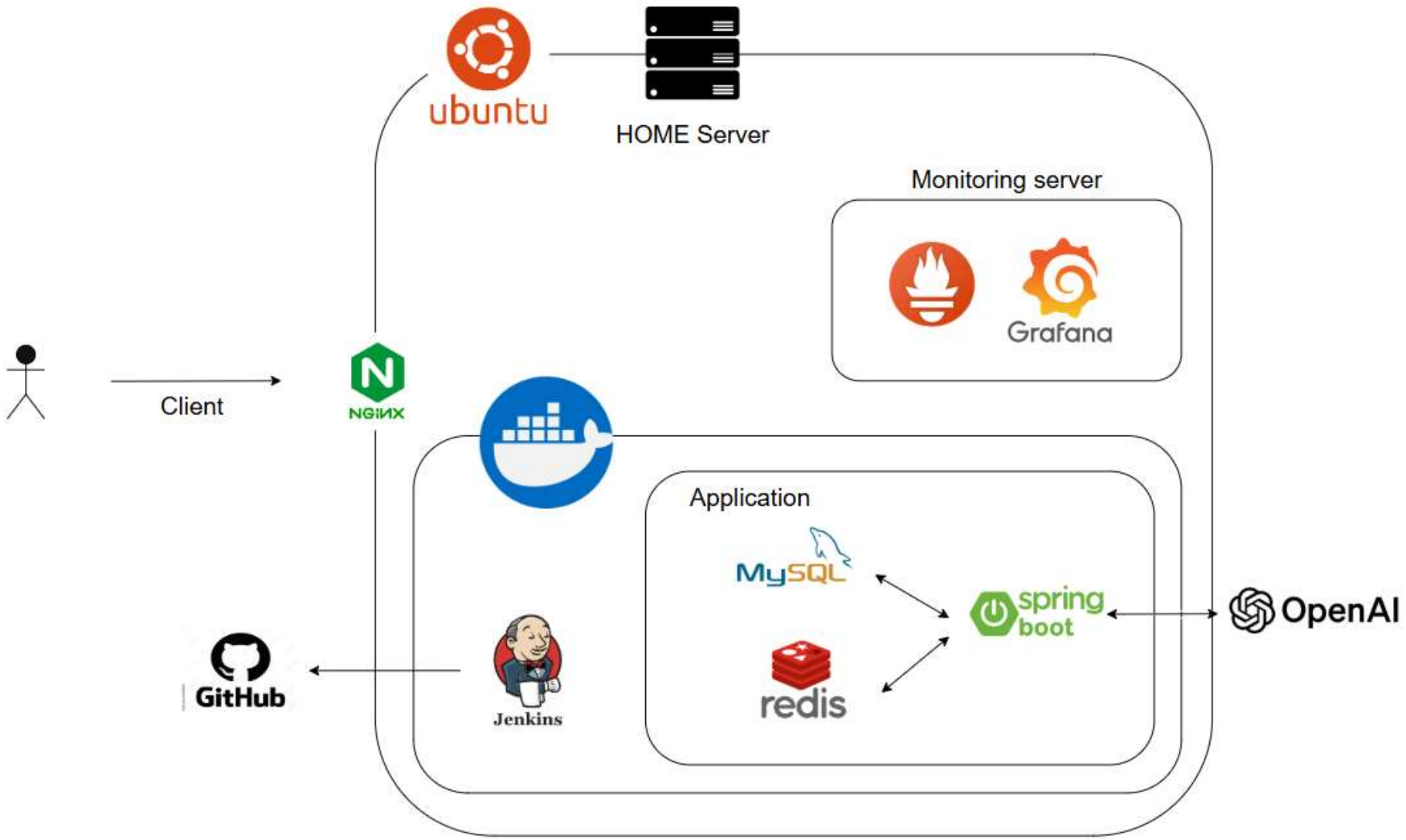
3. 어플리케이션

4. 주요 작업

■ HRMS – 프로젝트 아키텍처

환경 구성

- Ubuntu 기반 Home Server 환경에서 Docker를 활용하여 전체 서비스를 컨테이너 단위로 운영
- Nginx를 Reverse Proxy로 구성하여 외부 요청을 Spring Boot 애플리케이션으로 라우팅
- Redis를 활용하여 채팅 데이터 캐싱 및 실시간 처리 성능 최적화
- OpenAI API를 연동하여 AI 기반 근태 분석 기능 구현 (외부 API 호출 구조)
- Prometheus + Grafana를 활용하여 서버 리소스 및 API 응답 성능 모니터링 구축
- Jenkins를 이용한 CI/CD 파이프라인 구성으로 GitHub 코드 변경 시 자동 빌드 및 배포

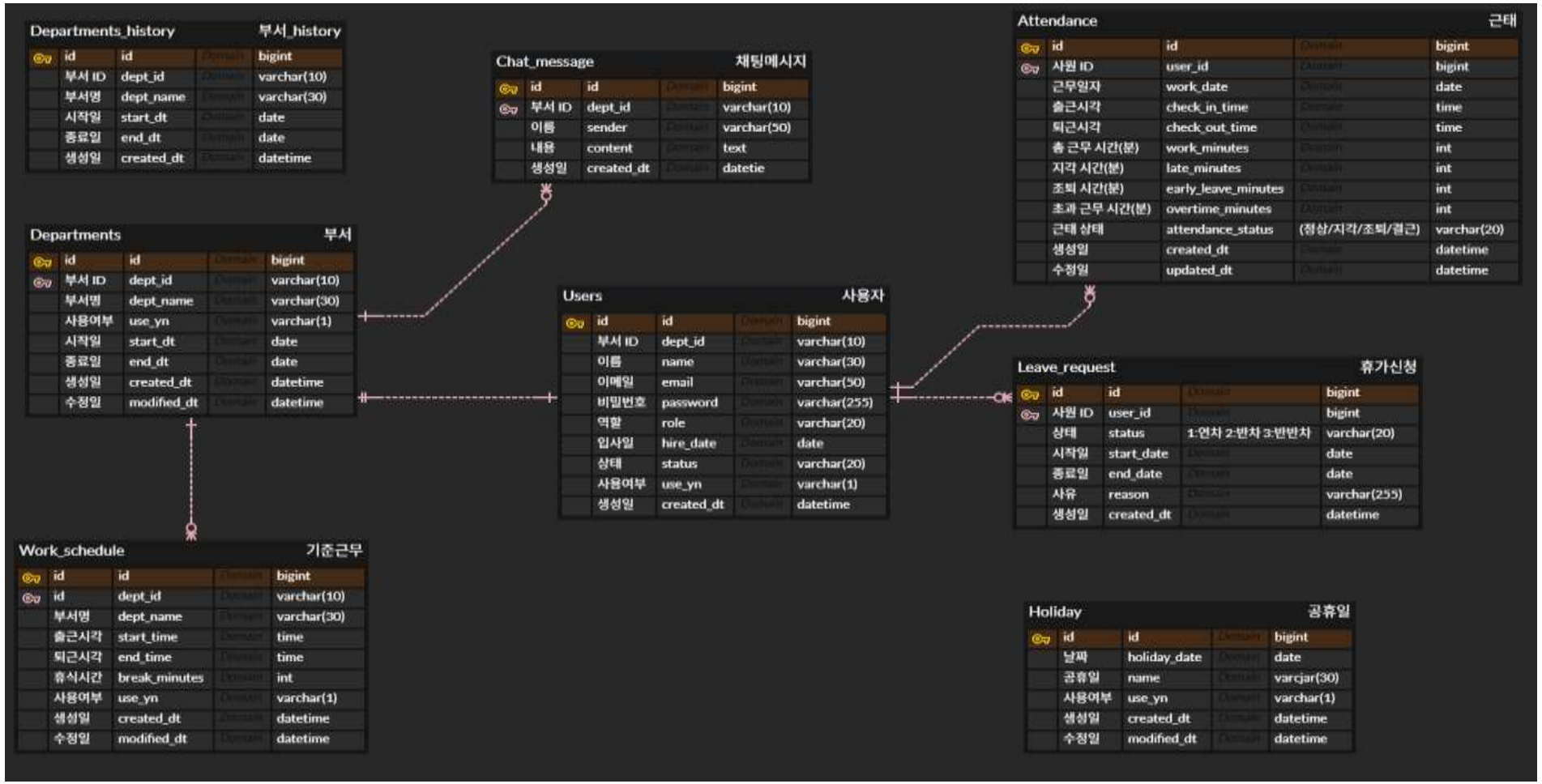


■ HRMS – 백엔드 CI/CD

목차

1. 주요 기능 요약
2. 배포
3. **어플리케이션**
4. 주요 작업

■ HRMS – Database



설계의도

- 사용자, 근태, 부서 등 핵심 도메인을 기준으로 Entity를 분리하여 관리
- 데이터 중복을 최소화하기 위해 정규화 기반의 관계형 구조 설계
- Entity 간 관계를 1:N 중심으로 설계하여 구조 단순화 및 유지보수 용이성 확보
- 근태 데이터는 이력 관리가 가능하도록 별도 테이블로 분리
- 공통 Audit 컬럼(created_dt, modified_dt)을 적용하여 데이터 추적 용이성 확보

목차

1. 주요 기능 요약

2. 배포

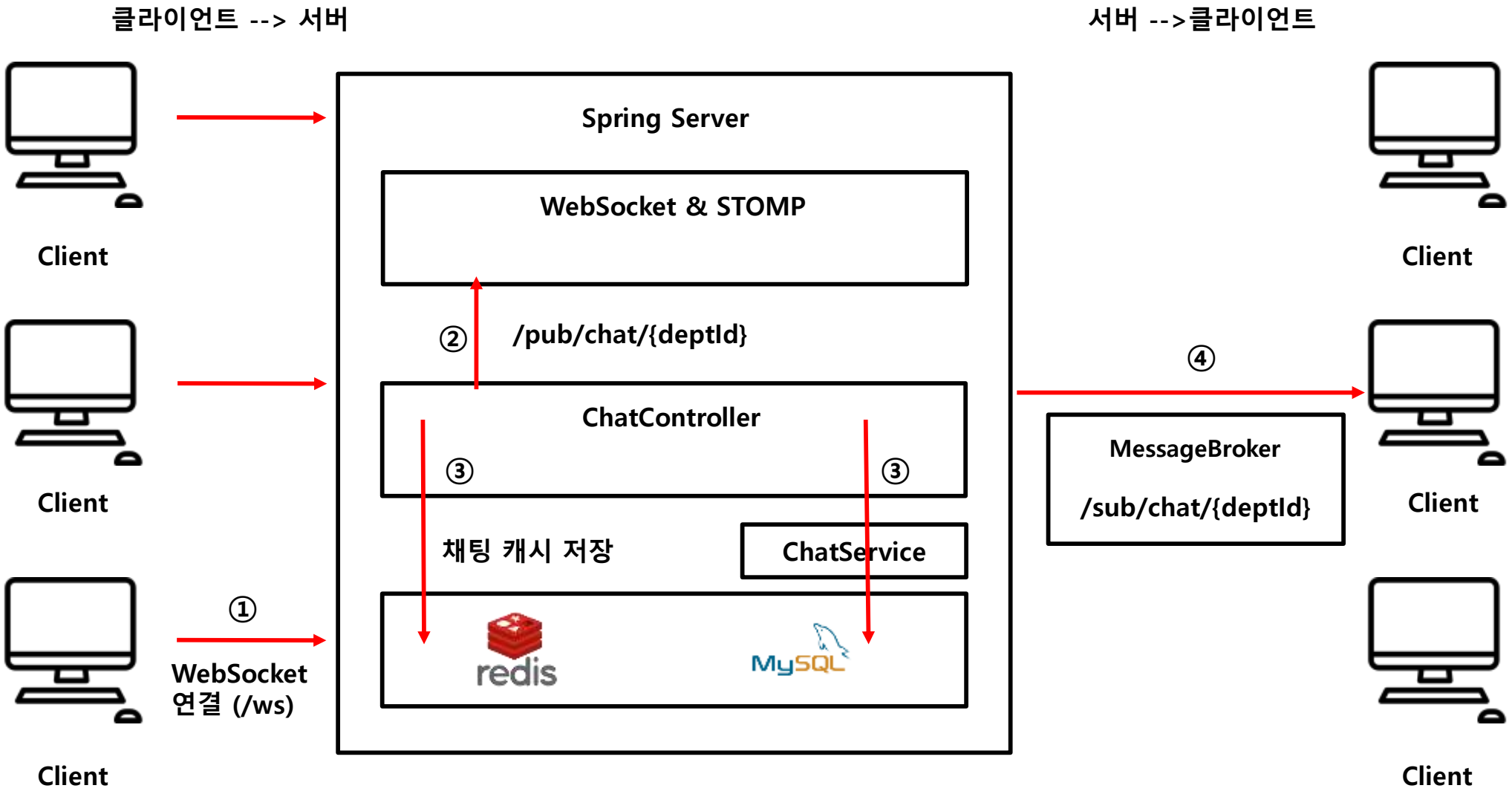
3. 어플리케이션

4. 주요 작업

1. 부서별 채팅
2. AI 근태분석
3. 모니터링 시스템

■ HRMS – 부서별 채팅_1

WebSocket + STOMP 기반 실시간 채팅 시스템을 구현하고 Redis 캐시를 활용한 조회 성능 개선과, DB(MySQL)를 통한 데이터 영속성을 확보하도록 설계



■ HRMS – 부서별 채팅_2

```java
@Configuration  wjkim0331@naver.com <wjkim0331@naver.com> +1
//WebSocket + STOMP 활성화
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    //pub : 클라이언트 -> 서버(클라이언트가 메시지 보낼 때 /pub로 시작)
    //sub : 서버 -> 클라이언트
    @Override  0개의 사용위치  wjkim0331@naver.com <wjkim0331@naver.com>
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker( ...destinationPrefixes: "/sub");
        registry.setApplicationDestinationPrefixes("/pub");
    }

    @Override  0개의 사용위치  KWJ +1
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        //웹소켓 접속 주소
        registry.addEndpoint( ...paths: "/ws")
                //CORS 허용
                .setAllowedOriginPatterns("*")
                .withSockJS();
    }
```

- WebSocket 엔드포인트(/ws) 연결
- /pub : 클라이언트 → 서버 채팅 전송
- /sub : 서버 → 클라이언트 채팅 전송

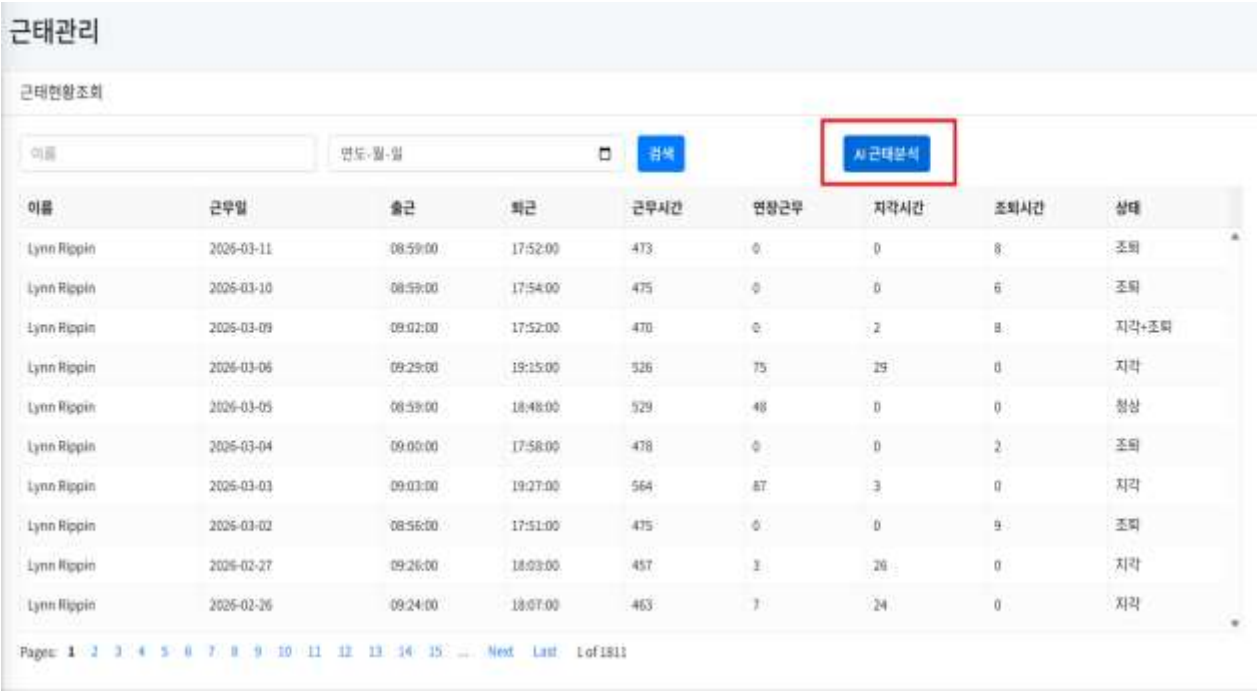
```java
@MessageMapping("/chat/{deptId}")  KWJ *
public void sendMessage(@DestinationVariable String deptId,
                        ChatMessage message) {

    message.setDeptId(deptId);

    // 1. DB 저장
    chatService.saveMessage(deptId, message);
    // 2. 실시간 전송
    messagingTemplate.convertAndSend( destination: "/sub/chat/" + deptId, message);
}
```

```java
@CacheEvict(value = "chat", key = "#deptId")  1개 사용 위치  KWJ
public void saveMessage(String deptId, ChatMessage message) {
    chatRepository.save(message);
}
```

- WebSocket 메시지를 수신하여 Redis에 저장

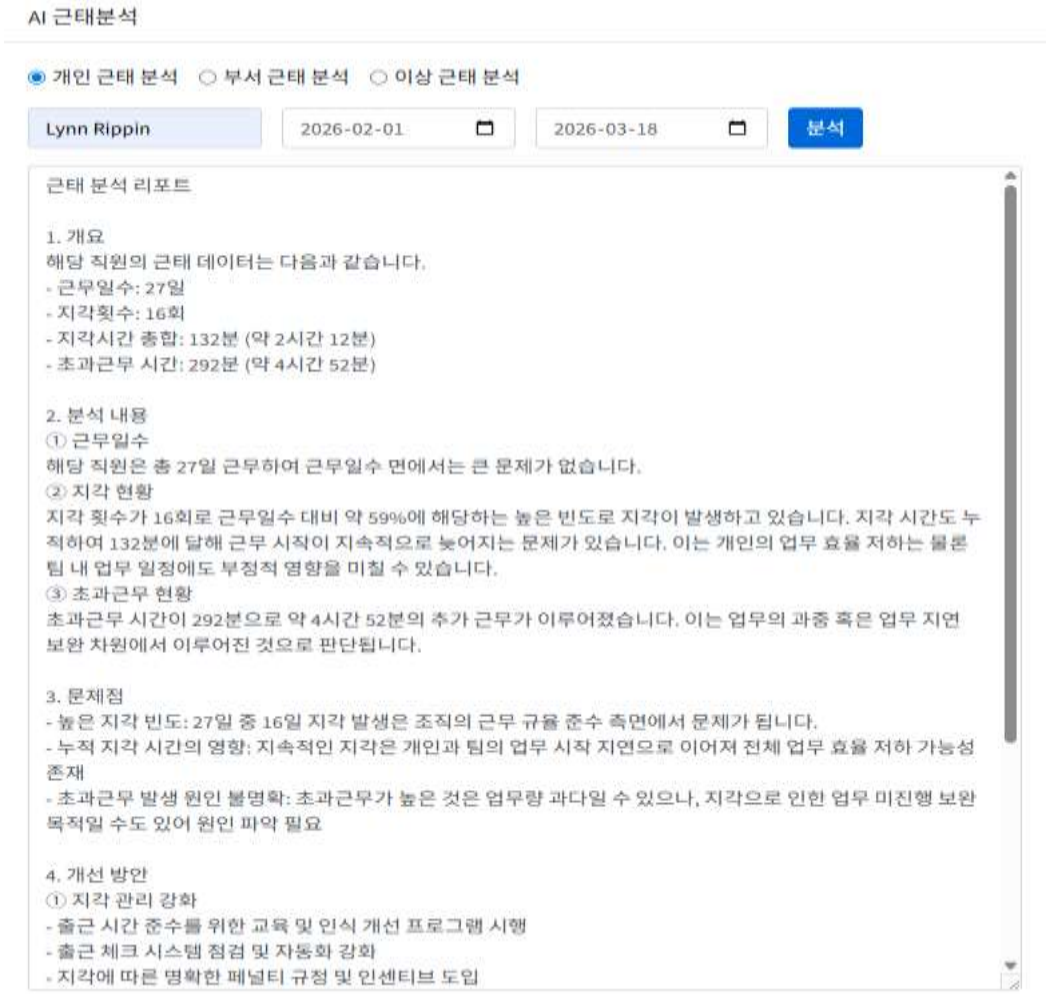
■ HRMS – AI 근태분석_1

개인/부서/이상 근태 분석에 필요한 데이터를 조회하여 프롬프트 작성 후, OpenAI API 호출을 통해 근태 분석 결과를 생성할 수 있도록 설계



이름	근무일	출근	퇴근	근무시간	연장근무	지각시간	조퇴시간	상태
Lynn Rippin	2026-03-11	08:59:00	17:52:00	473	0	0	8	조퇴
Lynn Rippin	2026-03-10	08:59:00	17:54:00	475	0	0	6	조퇴
Lynn Rippin	2026-03-09	09:02:00	17:52:00	470	0	2	8	지각+조퇴
Lynn Rippin	2026-03-06	09:29:00	19:15:00	526	75	29	0	지각
Lynn Rippin	2026-03-05	08:59:00	18:48:00	529	48	0	0	정상
Lynn Rippin	2026-03-04	09:00:00	17:58:00	478	0	0	2	조퇴
Lynn Rippin	2026-03-03	09:03:00	19:27:00	564	87	3	0	지각
Lynn Rippin	2026-03-02	08:56:00	17:51:00	475	0	0	9	조퇴
Lynn Rippin	2026-02-27	09:26:00	18:03:00	457	3	26	0	지각
Lynn Rippin	2026-02-26	09:24:00	18:07:00	463	7	24	0	지각

Pages: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 ... Next Last 1 of 1811



■ HRMS – AI 근태분석_2

```java
public String perAnalysis(AnalysisRequest request){
    AttendanceSummaryDTO data =
            attendanceRepository.getPersonAttendance(
                    request.getName(),
                    request.getDateFrm(),
                    request.getDateTo()
            );

    // 2 AI 프롬프트 생성
    String prompt = buildPerPrompt(data);

    // 3 GPT 호출
    return openAiService.callGPT(prompt);
      return prompt;
}
```

```java
private String buildPerPrompt(AttendanceSummaryDTO data){

    return """
        다음은 직원 근태 데이터이다.

        근무일수: %d
        지각횟수: %d
        지각시간: %d분
        초과근무: %d분

        위 데이터를 기반으로 근태 분석 리포트를 작성해라.
        문제점과 개선점도 포함해라.
    """.formatted(
            data.getWorkDays(),
            data.getLateCount(),
            data.getLateMinutes(),
            data.getOvertimeMinutes()
    );

}
```

- data에 필요 데이터 담은 후 promt로 전달

```java
@Service
@RequiredArgsConstructor
public class OpenAiService {

    @Value("${openai.api-key}")
    private String apiKey;

    private final RestTemplate restTemplate = new RestTemplate();

    public String callGPT(String prompt) {

        String url = "https://api.openai.com/v1/chat/completions";

        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_JSON);
        headers.setBearerAuth(apiKey);

        Map<String, Object> body = new HashMap<>();
        body.put("model", "gpt-4.1-mini");

        List<Map<String, String>> messages = new ArrayList<>();

        Map<String, String> msg = new HashMap<>();
        msg.put("role", "user");
        msg.put("content", prompt);

        messages.add(msg);
```

```java
        body.put("messages", messages);

        HttpEntity<Map<String, Object>> request =
                new HttpEntity<>(body, headers);

        ResponseEntity<Map> response =
                restTemplate.postForEntity(url, request, Map.class);

        List<Map<String, Object>> choices =
                (List<Map<String, Object>>) response.getBody().get("choices");

        Map<String, Object> choice = choices.get(0);
        Map<String, Object> message =
                (Map<String, Object>) choice.get("message");

        return (String) message.get("content");
    }

}
```

- Chat Completions API 호출
- messages 기반 요청 구성
- choices → message → content 파싱
- 최종 결과 반환

■ HRMS – 모니터링시스템_1

Prometheus와 Grafana를 활용하여 애플리케이션의 성능 지표를 수집하고 API 응답 시간 및 시스템 상태를 실시간으로 모니터링하도록 구성

- Spring Boot Actuator + Micrometer 기반 메트릭 수집
- Prometheus를 통해 메트릭 스크래핑
- Grafana를 활용한 시각화 및 분석

주요 수집 지표

1. API 응답 시간 (http_server_requests)
2. 요청 수 / 처리량
3. JVM 메모리 / CPU 사용량
4. 에러율 및 상태 코드 분포